Rishabh Indoria
24002
M.Tech AI

Machine Learning Signal Processing

April 12, 2025

1 Introduction

- **Text Data**: Represented as a sequence of UNICODE symbols.
- **UNICODE**: an international encoding standard for use with different languages and scripts, by which each letter, digit, or symbol is assigned a unique numeric value that applies across different platforms and programs.
- **UTF-8**: a standard that is most commonly used for encoding text. The first 128 symbols are ASCII characters and can encode more than 1M characters with variable width.
- **Speech Data**: Represented by phonetic units, underlying hidden variables. Data represented as a scalar series of 16-bit integers with a specific sampling rate (typically 16kHz).
- **Image Data**: Basic unit is a pixel.
- **Video Data**: Series of images sampled at a frame rate (e.g. 30Hz), can have an additional audio channel (sampled at higher rate).
- 3 Assignments(15), Midterm(20), Project(25), Finals(40).
- "Pattern Recognition and Machine Learning" by Bishop
- "Neural Networks" by Bishop
- "Deep Learning" by Good fellow
- Course Web page is `http://www.leap.ee.iisc.ac.in/sriram/teaching/MLSP25`
- Midterm on February second half, February 4th week project team and proposal finalization.
- March 3rd week project midterm presentation, April last week final presentation.
- April 3rd week final exams.

2 Useful Stuff

- Some useful derivatives
- $f(x) = x^T Ax$ w.r.t x, $\frac{\partial f}{\partial x} = (A + A^T)x$
- $f(x) = x^T Ax$ w.r.t A, $\frac{\partial f}{\partial A} = xx^T$
- $f(A) = Tr(A)$ w.r.t A, $\frac{\partial f}{\partial A} = I$
- $f(A) = Tr(AB)$ w.r.t A, $\frac{\partial f}{\partial A} = B^T$
- $f(A) = Tr(ABA^T)$ w.r.t A, $\frac{\partial f}{\partial A} = 2AB$
- $f(A) = \log |A|$ w.r.t A, $\frac{\partial f}{\partial A} = (A^{-1})^T$
- A twice differentiable function is convex if $f^{"}(x) \geq 0$ for all x.
- A function $f(x)$ is convex if for any x_1 and x_2 the following holds

$$f(\lambda x_1 + (1-\lambda)x_2) \leq \lambda f(x_1) + (1-\lambda)f(x_2), \ \forall \lambda \in [0,1]$$

- **Jensen's Inequality**: For any convex function f, the following holds

$$f(\sum_{i=1}^{n} \lambda_i x_i) \leq \sum_{i=1}^{n} \lambda_i f(x_i)$$

3 Dimensionality Reduction

3.1 Problem

- Reducing the data x_n of dimension D to lower dimension.
- $x_1, ..., x_n \in \mathbb{R}^{N\times 1} \to y_1, ..., y_n \in \mathbb{R}^{M\times 1}$ where $M < N$
- In general, we want $M << N$.

3.2 Principal Component Analysis

- Reference: PRML section 12.1
- This is a linear transform.
- **Objective**: Projecting the data into subspace which preserves maximum data variance
- Maximize variance in projected space $M < D$
- $y_n = A(x_n - b)$ where $x_n, b \in \mathbb{R}^{D\times 1}$, $A \in \mathbb{R}^{M\times D}$, $y_n \in \mathbb{R}^{M\times 1}$
- We need to determine A and b.
- Consider $M = 1$, $A \in \mathbb{R}^{1\times D}$, $A = u^T$

$$\text{Mean of } y = \bar{y} = \frac{1}{N}\sum_{i=1}^{N} u^T(x_i - b) = u^T\bar{x} - u^T b$$

$$\text{Variance of } y = S_y = \frac{1}{N}\sum_{i=1}^{N}(y_i - \bar{y})^2 = \frac{1}{N}\sum_{i=1}^{N} u^T(x_i - \bar{x})(x_i - \bar{x})^T u = u^T S_x u$$

- Take u as a unit vector, so variance gets bounded.
- If we don't then, by taking u as ∞ we can get ∞ variance.

$$\begin{aligned}
&\max S_y = u^T S_x u \text{ such that } u^T u = 1\\
&L(u, \lambda) = u^T S_x u + \lambda(1 - u^T u)\\
&\frac{\partial L}{\partial u} = (S_x + S_x^T)u + \lambda(-2u) = 2S_x u - 2\lambda u = 0\\
&\implies S_x u = \lambda u\\
&\implies S_y = u^T S_x u = u^T \lambda u = \lambda u^T u = \lambda\\
&\text{Maximize } S_y \to \text{Maximum eigenvalue of } S_x
\end{aligned}$$

- Suppose we have constructed k dimensions, k-top eigenvalues, $A^T = [u_1, ..., u_k]$
- Now we add a new dimension u_j such that $u_j \perp u_1, ..., u_k$

$$\begin{aligned}
&y_n = A(x_n - b) \text{ we want to maximize } Tr(S_y)\\
&Tr(S_y) = u_1^T S_x u_1 + u^T S_x u_2 + ... + u_k^T S_x u_k + u_j S_x u_j^T\\
&\text{such that } u_j^T u_j = 1 \text{ and } u_j^T u_l = 0\\
&L(u_j, \lambda, \mu_1, ..., \mu_k) = \sum_{l=1}^{k} u_l^T S_x u_l + u_j^T S_x u_j + \lambda(1 - u_j^T u_j) + \sum_{l=1}^{k} \mu_l u_j^T u_l\\
&\frac{\partial L}{\partial u_j} = (S_x + S_x)^T u_j - 2\lambda u_j + \sum_{l=1}^{k} \mu_l u_l = 0\\
&\text{Taking inner product with } u_j\\
&= u_j^T S_x u_j - \lambda = 0 \implies u_j^T S_x u_j = \lambda\\
&u_j \text{ is an eigenvector of } S_x\\
&\text{To be highest } \lambda = (k+1)^{th} \text{ highest eigenvalue}
\end{aligned}$$

- $A^T = [u_1, ..., u_M]$, PCA is defined as $y_n = A(x_n - b)\ \forall n$

- $u_1, ..., u_M$ correspond to M "top" eigenvalues of S_x
- b can be anything, most common is $\bar{x}$, aka **centering**.
- Centering makes data have zero mean.
- Equivalent, formulated as minimizing the error between the original and projected data points.
- Let $A^T = [u_1, ..., u_n]$, $u_j's$ are orthogonal

  Basis for D dimensions, $\{u_i\}_{i=1}^D$

  Any data $x_n = \sum_{i=1}^{D} \alpha_n^i u_i,\ \alpha_n^i = x_n^T u_i$

  $x_n \in \mathbb{R}^{D\times 1} \to \tilde{x}_n$ lies in span of M vectors

  Without loss of generality $\tilde{x}_n = \sum_{i=1}^{M} z_n^i u_i + c,$ where $c = \sum_{i=M+1}^{D} b_i u_i,\ b_i$ are same for all points.

  Unknowns: u_i, z_n^i, b_i

  We want to minimize $J_N = \frac{1}{N}\sum_{n=1}^{N} \|x_n - \tilde{x}_n\|^2$

  Putting $\frac{\partial J}{\partial z_n^J} = 0 \implies z_n^J = \alpha_n^j$ and $\frac{\partial J}{\partial b_j} = 0 \implies b_j = \bar{x}^T u_j$

  Substituting these we get

  $J_N = \frac{1}{N}\sum_{n=1}^{N} \| \sum_{i=M+1}^{D} (x_n - \bar{x})^T u_i \| = \frac{1}{N}\sum_{n=1}^{N}\sum_{i=M+1}^{D} u_i^T (x_n - \bar{x})(x_n - \bar{x})^T u_i = \sum_{M+1}^{D} u_i^T S_x u_i$

  Minimize $\to \{u_i\}_{i=M+1}^D$ are eigenvectors of S_x corresponding to least eigenvalues

  $\{u_i\}_{i=1}^M \to$ eigenvectors corresponding to max eigenvalues/remaining eigenvalues

- Whenever we go from higher to lower dimension, we are bringing the data points closer.
- We try to preserve as much of the separability/variance of the data.
- **Standardization**: Make variance 1 along each axis and mean 0.
- **Whitening**: Do standardization and make all covariance 0, makes sure variables are uncorrelated with each other.

  Transform $x_n \to y_n$ such that $\bar{y} = 0 \& S_y = I$

  Extension of PCA $M = D$, $y_n = A(x_n - \bar{x})$

  This gives $\bar{y} = 0,$ and $S_y = \Lambda$ (Eigenvalue matrix)

  So if we take $y_n = \Lambda^{-\frac{1}{2}} A(x_n - \bar{x}) \implies S_y = I$

- For $D \times D$ matrix, eigenvalue computation is $O(D^3)$
- For high dimensional data, $D >> N$, this becomes infeasible.
- If we create a data matrix, $X = [(x_1 - \bar{x}), ..., (x_n - \bar{x})]$ then , $S_x = \frac{1}{N} X^T X$.

  v_i is a normalized eigenvector of $\frac{1}{N} X X^T$

  $\frac{1}{N} X X^T v_i = \lambda_i v_i$

  Goal: Find u_i such that $\frac{1}{N} X^T X u_i = \lambda_i u_i$

  Multiply X^T in first equation

  $\frac{1}{N}(X^T X)(X^T v_i) = \lambda_i (X^T v_i)$

  $u_i \propto X^T v_i \implies \langle X^T v_i, X^T v_i \rangle = v_i^T X X^T v_i = v_i^T N \lambda_i v_i = N\lambda_i$

  $u_i = \frac{1}{\sqrt{N\lambda_i}} X^T v_i,$ however the eigenvalues are the same

3.3 Linear Discriminant Analysis

- Reference: PRML section 4.1.4
- Let's say we have additional information about the data, specifically the class data.
- In this case, PCA would not be able to incorporate this.
- **Given**: $\{x_n\}_{i=1}^N$ data points and number of classes k
- Assume $k = 2$, i.e., $x_n \in \{C_1, C_2\}$, we first define class means

$$m_1^x = \frac{1}{N_1} \sum_{x \in C_1} x_n \text{ and } m_2^x = \frac{1}{N} \sum_{x \in C_2} x_n, \text{ where } N_1 = |C_1|, N_2 = |C_2|$$

- **Objective**: $y = w^T x$ such that m_1^y and m_2^y are maximally apart, w is a unit vector.

$$\begin{aligned}
&\max(m_1^y - m_2^Y)^2 = (w^T(m_1^x - m_2^x)) \text{ such that } \|w\| = 1 \\
&S_B^x = (m_1^x - m_2^x)(m_1^x - m_2^x)^T = \text{ Between class covariance/Scatter Matrix} \\
&w \propto (m_1^x - m_2^x) \\
&S_W^x = \sum_{n \in C_1} (x_n - m_1^x)(x_n - m_1^x)^T + \sum_{n \in C_2} (x_n - m_2^x)(x_n - m_2^x)^T = \text{ Within class covariance}
\end{aligned}$$

- We define the **Fischer Discriminant**

$$J = \frac{w^T S_B^x w}{w^T S_W^x w} \text{ maximize w.r.t } w \text{ such that } \|w\| = 1$$

- w is eigenvector of $(S_W^{-1} S_B)$, generalized eigenvectors.
- In general, $S_B = \sum N_k (m_k - m)(m_k - m)^T$, where $m =$ global mean
- $rank(S_B) \leq k - 1$ which means $S_W^{-1} S_B$ also has $\leq k - 1$ rank
- This implies that there are $\leq k - 1$ eigenvectors, rest are 0.
- Take $M \leq k - 1$ for meaningful LDA.
- Multi class multidimensional Fischer is defined as

$$J = ((w^T S_W w)^{-1} (w^T S_B w))$$

4 Decision Theory

4.1 Introduction

- Reference: PRML section 1.5
- Without loss of generalization, assume number of classes $k = 2$.
- Data is $x_n \in \mathbb{R}^d$, and $t_n \to$ target label.
- We are going to consider x_n, t_n as random variable.
- **Holy Grail**: $P(x, t)$, if we have this then we can find class posterior, $P(t|x)$ distribution.

$$P(t|x) = \frac{P(x|t)P(t)}{P(x)}$$

$$P(x): \text{ Data Probability} \quad P(x|t): \text{ Class Collection Likelihood} \quad P(t): \text{ Prior Probability}$$

- We will assume classifier is naive, **Decision Rule**.
- Take $x \in \mathbb{R}_1 \to C_1$ and $x \in \mathbb{R}_2 \to C_2$
- **Evaluation Objective**: Minimization of Error

$$P(\text{Error}) = P(x \in \mathbb{R}_1, C_2) + P(x \in \mathbb{R}_2, C_1)$$

- We take $x \in C_k$ if

$$P(x, C_k) > P(x, C_j) \implies P(C_k|x) > P(C_j|x) \forall j, j \neq k$$
$$k = \arg\max_j P(C_j|x)$$

- This is called **Maximum A-posterior Rule (MAP)**.
- Approaches for Inference and Decision
  1. **Generative Modeling**: Finding the joint density from the data, $P(x|C_k)P(C_k)$
  2. **Discriminative Modeling**: Finding the posteriors directly, $P(C_k|x) \propto P(x|C_k)P(C_k)$
  3. Using discriminant functions for classification, $P(x|C_k) \sim N$
- For Regression, we minimize mean squared error loss, solution is conditional expectation.

4.2 Generative Modeling

- Collection of probability distributions which are described by a **finite dimensional parameter set**.
- **Non-parametric** models do not specify an apriori set of parameters to model the distribution.
- Non-parametric methods are dependent on number of data points, estimation is difficult for **large datasets**.
- **Likelihood computation** and model comparisons are hard.
- **Limited use** in classifiers.
- Example: Kernel Density Estimators.
- Parametric Modeling Reference: PRML chapter 2
- Collection of probability distributions which are described by a **finite dimensional parameter set**.
- Parameters $\boldsymbol{\theta} = (\theta_1, \theta_2, ..., \theta_K)$ for probability distributions $\boldsymbol{P} = \{P_{\boldsymbol{\theta}}\}$

$$\text{Poisson Distribution: } p_\lambda(j) = \frac{\lambda^j}{j!} e^{-\lambda}$$
$$\text{Bernoulli Distribution: } p(\boldsymbol{x}|\boldsymbol{\mu}) = \prod_{i=1}^{D} \mu_i^{x_i} (1 - \mu_i)^{x_i}$$
$$\text{Gaussian Distribution: } p(\boldsymbol{x}|\boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{\sqrt{(2\pi)^D |\boldsymbol{\Sigma}|}} \exp\{-\frac{1}{2}(\boldsymbol{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\boldsymbol{x} - \boldsymbol{\mu})\}$$

- Gaussian Distribution is one of the **most widely used and well studied model**.
- Points of equal probability lie on one contour.
- First, we will fit the data with a Gaussian model, points of each class are fit with their own model.
- We need to find the following parameters

$$\boldsymbol{\theta} = (\boldsymbol{\mu}, \boldsymbol{\Sigma})$$

- Total number of parameters to be learned for D dimensional data is $\frac{D(D+1)}{2} + D$
- Given N data points of class k $\{x_i\}_{i=1}^N$, we need to estimate the parameters of the model.
- Several criteria can be used, most popular is the **maximum likelihood estimation (MLE)**.
- The likelihood function is defined as $L(\boldsymbol{\theta}) = \prod_{i=1}^N p(\boldsymbol{x}_i|\boldsymbol{\theta})$
- To avoid multiplication, we generally take natural log of the above.
- The **maximum likelihood estimator** is

$$\boldsymbol{\theta}^* = \arg\max_{\boldsymbol{\theta}} L(\boldsymbol{\theta})$$

- The MLE satisfies some **nice properties** like
  1. **Consistency**, convergence to true value.

2. **Efficiency**, has the least mean squared error.

- We defined the Gaussian distribution above, now we input that in MLE and take natural log

$$\log L(\boldsymbol{\theta}) = -\frac{ND}{2} - \frac{N}{2}\log|\boldsymbol{\Sigma}| - \frac{1}{2}\sum_{i=1}^{N}((\boldsymbol{x}_i - \boldsymbol{\mu})^T\boldsymbol{\Sigma}^{-1}(\boldsymbol{x}_i - \boldsymbol{\mu}))$$

- We estimate the parameters by taking derivative with respect to the required parameters.

$$\frac{\partial L}{\partial \boldsymbol{\mu}} = \mathbf{0}_{D\times 1} \implies \boldsymbol{\mu} = \frac{1}{N}\sum_{i=1}^{N}\boldsymbol{x}_i$$

$$\frac{\partial L}{\partial \boldsymbol{\Sigma}} = \mathbf{0}_{D\times D} \implies \boldsymbol{\Sigma}^{-1} = \frac{1}{N}\sum_{i=1}^{N}(\boldsymbol{x}_i - \boldsymbol{\mu})(\boldsymbol{x}_i - \boldsymbol{\mu})^T$$

- First one is straightforward, second one can be derived using $\frac{\partial L}{\partial \boldsymbol{\Sigma}^{-1}} = \mathbf{0}$
- Sample mean and covariance are MLE of parameters of an underlying Gaussian distribution.
- However, it is possible that fitting a single Gaussian model may be **too broad**.
- To solve this, we need **mixture models**. These can fit any **arbitrary distribution**.
- A **Gaussian Mixture Model (GMM)** is defined as

$$p(\boldsymbol{x}|\boldsymbol{\Theta}) = \sum_{k=1}^{K}\alpha_k p(\boldsymbol{x}|\boldsymbol{\theta}_k)$$

- $p(\boldsymbol{x}|\boldsymbol{\theta}_k)$ are Gaussian
- The weighing coefficients have the property.

$$\sum_{k=1}^{K}\alpha_k = 1$$

- GMMs can model multi-modal data, identify data clusters, can model arbitrarily complex data distribution.
- The set of parameters for the model are

$$\boldsymbol{\Theta}_K = \{\alpha_k, \boldsymbol{\theta}_k\}_{k=1}^{K} \quad \boldsymbol{\theta}_k = \{\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\}$$

- The number of parameters are $KD^2 + KD + K$
- The log-likelihood function over the entire data in this case will have a **logarithm of a summation**.

$$\log L(\boldsymbol{\Theta}) = \sum_{i=1}^{N}\log\left(\sum_{k=1}^{K}\alpha_k p(\boldsymbol{x}_i|\boldsymbol{\theta}_k)\right)$$

- Solving for the optimal parameters using MLE for GMM is not straight forward.
- Resort to the **Expectation Maximization (EM)** algorithm.
- We will do iterative optimization

$$\boldsymbol{\Theta}_n \to \boldsymbol{\Theta}_{n+1} \text{ such that } L(\boldsymbol{\Theta}_n) \leq L(\boldsymbol{\Theta}_{n+1})$$

- EM algorithm introduces a hidden discrete variable.
- Let $\boldsymbol{z}$ be that discrete-hidden variable.

$$p(\boldsymbol{x}|\boldsymbol{\Theta}) = \sum_{\boldsymbol{z}} p(\boldsymbol{x}|\boldsymbol{z}, \boldsymbol{\Theta})p(\boldsymbol{z}|\boldsymbol{\Theta})$$

$$
\begin{aligned}
L(\boldsymbol{\Theta}) - L(\boldsymbol{\Theta}_n) &= \log p(\boldsymbol{x}|\boldsymbol{\Theta}) - \log p(\boldsymbol{x}|\boldsymbol{\Theta}_n) \\
&= \log \big(\sum_{\boldsymbol{z}} p(\boldsymbol{x}|\boldsymbol{z}, \boldsymbol{\Theta}) p(\boldsymbol{z}|\boldsymbol{\Theta})\big) - \log p(\boldsymbol{x}|\boldsymbol{\Theta}_n) \\
&= \log \big(\sum_{\boldsymbol{z}} p(\boldsymbol{x}|\boldsymbol{z}, \boldsymbol{\Theta}) p(\boldsymbol{z}|\boldsymbol{\Theta}) \cdot \frac{p(\boldsymbol{z}|\boldsymbol{x}, \boldsymbol{\Theta}_n)}{p(\boldsymbol{z}|\boldsymbol{x}, \boldsymbol{\Theta}_n)}\big) - \log p(\boldsymbol{x}|\boldsymbol{\Theta}_n) \\
&= \log \big(\sum_{\boldsymbol{z}} p(\boldsymbol{z}|\boldsymbol{x}, \boldsymbol{\Theta}_n) \cdot \frac{p(\boldsymbol{x}|\boldsymbol{z}, \boldsymbol{\Theta}) p(\boldsymbol{z}|\boldsymbol{\Theta})}{p(\boldsymbol{z}|\boldsymbol{x}, \boldsymbol{\Theta}_n)}\big) - \log p(\boldsymbol{x}|\boldsymbol{\Theta}_n) \\
&\quad \text{From Jensen's Inequality} \\
&\geq \sum_{\boldsymbol{z}} p(\boldsymbol{z}|\boldsymbol{x}, \boldsymbol{\Theta}_n) \cdot \log \big(\frac{p(\boldsymbol{x}|\boldsymbol{z}, \boldsymbol{\Theta}) p(\boldsymbol{z}|\boldsymbol{\Theta})}{p(\boldsymbol{z}|\boldsymbol{x}, \boldsymbol{\Theta}_n)}\big) - \log p(\boldsymbol{x}|\boldsymbol{\Theta}_n) \\
&= \sum_{\boldsymbol{z}} p(\boldsymbol{z}|\boldsymbol{x}, \boldsymbol{\Theta}_n) \cdot \log \big(\frac{p(\boldsymbol{x}|\boldsymbol{z}, \boldsymbol{\Theta}) p(\boldsymbol{z}|\boldsymbol{\Theta})}{p(\boldsymbol{z}|\boldsymbol{x}, \boldsymbol{\Theta}_n) p(\boldsymbol{x}|\boldsymbol{\Theta}_n)}\big) \\
&\triangleq \Delta(\boldsymbol{\Theta}|\boldsymbol{\Theta}_n)
\end{aligned}
$$

- This means that
$$L(\boldsymbol{\Theta}) \geq L(\boldsymbol{\Theta}_n) + \Delta(\boldsymbol{\Theta}|\boldsymbol{\Theta}_n) = l(\boldsymbol{\Theta}|\boldsymbol{\Theta}_n)$$
- We now have a function, $l(\boldsymbol{\Theta}|\boldsymbol{\Theta}_n)$ which is bounded above by the likelihood function $L(\boldsymbol{\Theta})$
- Putting $\boldsymbol{\Theta} = \boldsymbol{\Theta}_n$ makes l equal to L.
- In order to achieve the greatest possible increase in the value of $L(\boldsymbol{\Theta})$, the EM algorithm calls for selecting $\boldsymbol{\Theta}$ such that $l(\boldsymbol{\Theta}|\boldsymbol{\Theta}_n)$ is maximized.

$$
\begin{aligned}
\boldsymbol{\Theta}_{n+1} &= \arg\max_{\boldsymbol{\Theta}} \{l(\boldsymbol{\Theta}|\boldsymbol{\Theta}_n)\} \\
&\text{Drop terms which are constant w.r.t. } \boldsymbol{\Theta} \\
&= \arg\max_{\boldsymbol{\Theta}} \{\sum_{\boldsymbol{z}} P(\boldsymbol{z}|\boldsymbol{X}, \boldsymbol{\Theta}_n) \log \big(P(\boldsymbol{X}|\boldsymbol{z}, \boldsymbol{\Theta}) P(\boldsymbol{z}|\boldsymbol{\Theta})\big)\} \\
&= \arg\max_{\boldsymbol{\Theta}} \{\sum_{\boldsymbol{z}} P(\boldsymbol{z}|\boldsymbol{X}, \boldsymbol{\Theta}_n) \log P(\boldsymbol{X}, \boldsymbol{z}|\boldsymbol{\Theta})\} \\
&= \arg\max_{\boldsymbol{\Theta}} \{E_{\boldsymbol{z}|\boldsymbol{X}, \boldsymbol{\Theta}_n} \{\log P(\boldsymbol{x}, \boldsymbol{z}|\boldsymbol{\Theta})\}\}
\end{aligned}
$$

- The hidden variables will be the index of the mixture component which generated.
- **Expectation Maximization Algorithm**
  1. Iterative procedure
  2. Assume the existence of hidden variable z_i associated with each data sample x_i
  3. Let the current estimate be Θ^n, define the Q function(auxiliary function) as
  $$
  \begin{aligned}
  Q(\Theta, \Theta^n) &= E_{z|X, \Theta^n} [\log \big(P(X, z|\Theta)\big)] \\
  &= \sum_z \log \big(P(X, z|\Theta)\big) P(z|X, \Theta^n)
  \end{aligned}
  $$
  4. It can be proven that if we choose
  $$\Theta^{n+1} = \arg\max_{\Theta} Q(\Theta, \Theta^n) = \arg\max_{\Theta} l(\Theta|\Theta^n)$$
  then $L(\Theta^{n+1}) \geq L(\Theta^n)$
  5. In many cases, finding the maximum for the Q function may be easier than likelihood function w.r.t. the parameters.
  6. Solution is dependent on finding a good choice of the hidden variables, which eases the computation.
  7. Iteratively improve the log-likelihood function.
- EM algorithm for GMM
  1. $z_i = \{1, ..., K\} \implies$ indicates the Gaussian in mixture from which data x_i is sampled.

2. $p(z_i = l) = \alpha_l$

$$l(\Theta|\Theta^n) = Q(\Theta, \Theta^n) \triangleq \sum_{i=1}^{N}\sum_{l=1}^{K} \log\left(P(x_i, z_i = l|\Theta)\right)P(Z_i = l|x_i, \Theta^n)$$

$$\text{We can write } P(x_i, z_i = l|\Theta) = P(z_i = l|\Theta) \cdot P(x_i|z_i = l, \Theta)$$

$$= \sum_{l=1}^{K}\sum_{i=1}^{N}(\log(\alpha_l) + \log(P_l(x_i|\Theta)))P(z_i = l|x_i, \Theta^n)$$

3. Initialize with a set of model parameters
4. Compute the conditional expectation(E-step)
5. Maximize the conditional expectation w.r.t. paramter(M-step)
6. Check for convergence
7. Go back to E-step if model has not converged
8. Re-estimation formulae can be derived by differentiating the above equation

$$\alpha_l^{new} = \frac{1}{N}\sum_{i=1}^{N} p(l|x_i, \Theta^n)$$

$$\mu_l^{new} = \frac{\sum_{i=1}^{N} x_i p(l|x_i, \Theta^n)}{\sum_{i=1}^{N} p(l|x_i, \Theta^n)}$$

$$\Sigma_l^{new} = \frac{\sum_{i=1}^{N} p(l|x_i, \Theta^n)(x_i - \mu_l^{new})(x_i - \mu_l^{new})^T}{\sum_{i=1}^{N} p(l|x_i, \Theta^n)}$$

- **GMM Application**: Generative modeling and Unsupervised clustering
- Mean initialization can be done randomly or by using k-means
- Covariance matrix can be kept constant(Identity), learn just diagonals or learn the entire matrix. Best representation will obviously be given by learning the entire matrix.

4.3 Function Modeling

- **Linear Models for Regression**: Chapter 3, PRML
- Linear Regression model is defined as

$$y(x, w) = w_0 + w^T x$$

- However this is too restrictive, we will instead use

$$y(x, w) = w_0 + \sum_{j=1}^{\mu-1} w_j \phi(x)$$

- Here ϕ is called a basis function, which is a non-linear fixed function.
- μ becomes a hyperparameter
- Polynomial regression uses a polynomial bases

$$y(x, w) = w_0 + w_1 x + w_2 x^2 + ... + w_M x^M = \sum_{j=0}^{M} w_j x^j$$

- Assume t is the target variable and $\epsilon \sim \mathcal{N}(0, \beta^{-1})$ is some Gaussian noise

$$t = y(x, w) + \epsilon$$

- We will consider the following underlying distribution

$$p(t|x, (w, \beta)) \sim \mathcal{N}(t|y(x, w), \beta^{-1})$$

$$L(t_1, ..., t_N|x_1, ..., x_N, (w, \beta)) = \prod_{n=1}^{N} \mathcal{N}(t_i|y(x_i, w), \beta^{-1})$$

$$\log L = \sum_{n=1}^{N} \frac{1}{2}\ln\beta - \frac{1}{2}\ln 2\pi - \frac{\beta}{2}(t_n - w^T\Phi(x_n))^2$$

- We only care about the last term, which is the mean-squared error.
- Consider that Φ gives M terms by inputting some x, we can write this as

$$\Phi(x) = \phi_0(x) + \phi_1(x) + ... + \phi_{M-1}(x)$$

  where each ϕ gives a single term
- So we can write this as

$$\Phi(x) = \begin{bmatrix} \phi_0(x) \\ \phi_1(x) \\ \vdots \\ \phi_{M-1}(x) \end{bmatrix}$$

- For N x_i's this can be written as

$$\Phi = \begin{bmatrix} \phi_0(x_1) & ... & \phi_{M-1}(x_1) \\ \vdots & & \\ \phi_0(x_N) & ... & \phi_{M-1}(x_N) \end{bmatrix}_{N \times M}$$

- Now we can differentiate log likelihood function to get

$$\begin{aligned} w_{ML} =& (\Phi^T \Phi)^{-1} \Phi^T t \\ (\beta_{ML})^{-1} =& \frac{1}{N} \sum_{n=1}^{N} \{t_n - w_{ML}^T \phi(x_n)\}^2 \end{aligned}$$

- **Online Learning**: Update model parameters incrementally
- Inverse calculation is costly, we would like an iterative learning process
- Let $E(w, \beta)$ be the least mean square error, we can update using gradient

$$w^{(z+1)} = w^{(z)} - \eta \frac{\partial E}{\partial w}$$

  where η is the learning rate
- As M increases the MSE, $E_D = \sum_n E_n(w)$, decreases due to "overfitting".
- To combat this, we can add a regularization term $E_W(w) = \frac{1}{2} w^T w$, we define regularized least squares

$$\begin{aligned} E(w) &= E_D(w) + \lambda E_W(w) \\ \hat{w}_{reg-ls} &= (\lambda I + \Phi^T \Phi)^{-1} \Phi^T t \\ L_q(w) &= (\sum_{j=1}^{M} |w_j|^q)^{\frac{1}{q}} \end{aligned}$$

- $E_W(w) = L_2(w)$ in the above but it can be anything

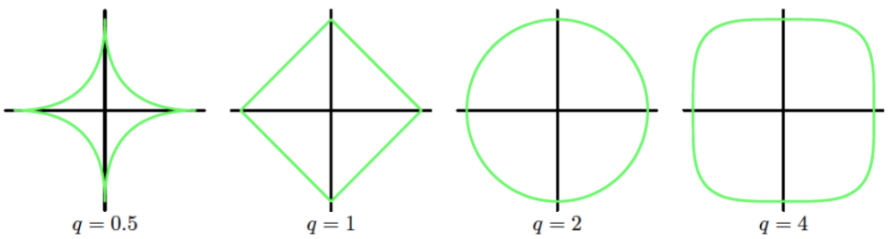


Figure 1: Regularization Term with different q

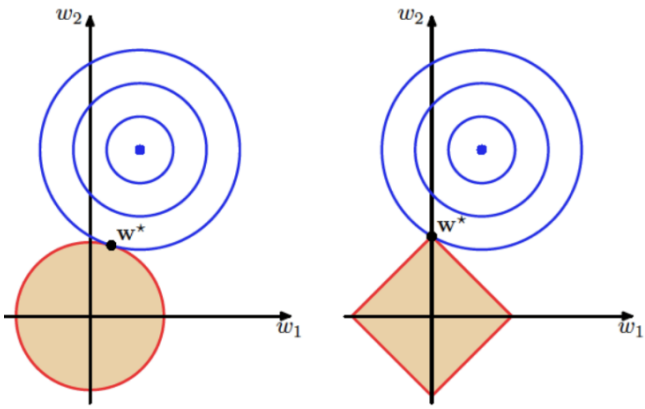


Figure 2: Resulting parameters with $q = 2$ and $q = 1$

- **Bias-Variance Trade-off**
- Consider the expectation of Likelihood

$$
\begin{aligned}
E[L] &= \int_t \int_x (y(x) - t)^2 p(x,t) dx dt \\
h(x) &= E_{t|x}[t] = \int t p(t|x) dt \\
E[L] &= \int (y(x) - h(x))^2 p(x) dx + \iint (h(x) - t)^2 p(x,t) dx dt + 2 \int ...
\end{aligned}
$$

- In an ideal world, we will have $h(x)$
- However, in the practical world we only have x's and t's
- Define something, $E_D(y(x;D))$

$$
\begin{aligned}
\{y(x;D) - h(x)\}^2 =& \{y(x;D) - E_D(y(x;D)) + E_D(y(x;D)) - h(x)\}^2 \\
=& \{y(x;D) - E_D(y(x;D))\}^2 + \{E_D(y(x;D)) - h(x)\}^2 + \\
& 2\{y(x;D) - E_D(y(x;D))\}\{E_D(y(x;D)) - h(x)\} \\
E_D[\{y(x;D) - h(x)\}^2] =& E_D[\{y(x;D) - E_D(y(x;D))\}^2] + \{E_D(y(x;D)) - h(x)\}^2 + \epsilon \\
=& \quad Variance \quad + \quad Bias^2 \quad + Noise
\end{aligned}
$$

- High regularization parameter leads to high bias and low variance
- Low regularization parameter leads to low bias and high variance

$$\text{expected loss} = (\text{bias})^2 + \text{variance} + \text{noise}$$

$$
\begin{aligned}
(\text{bias})^2 &= \int \{E_D[y(x;D)] - h(x)\}^2 p(x) dx \\
\text{variance} &= \int E_D[\{y(x;D) - E_D[y(x;D)]\}^2] p(x) dx \\
\text{noise} &= \int \{h(x) - t\}^2 p(x,t) dx dt
\end{aligned}
$$

- **Parameters**: Learnable from data directly
- **Hyper-parameters (fixed)**: Set based on held-out(validation) data.
- **Linear Models for Classification**
- Consider a 2 class classification problem, and let the model be $y(x) = w^T \Phi(x) + w_0$

$$
t = \begin{cases} C_1, & \text{for } y(x) \geq 0 \\ C_2, & \text{for } y(x) < 0 \end{cases}
$$

- Optimize a modified cost function, if we have a data point x

$$x = x_\perp + r\frac{w}{\|w\|}, \text{where } r \text{ is unknown}$$
$$y(x) = w^T x + w_0 = w^T(x_\perp + r\frac{w}{\|w\|}) = r\|w\|$$
$$r = \frac{y(x)}{\|w\|} = \text{signed distance}$$

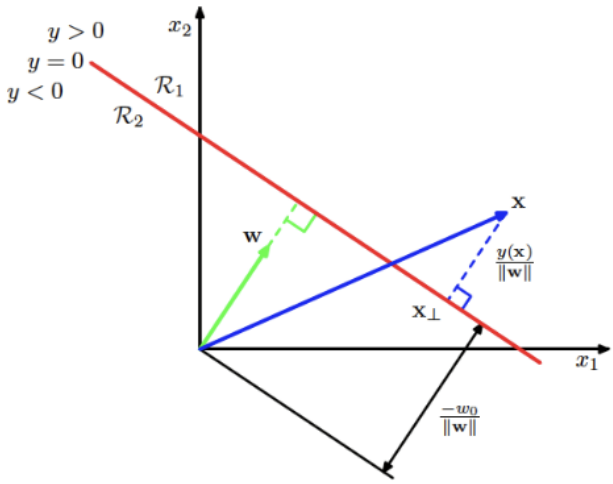


Figure 3: Optimize a modified cost function

- Now consider K classes, we learn $y_k(x) = w_k^T x + w_{k0}$ functions.
- Decide $x \in C_j$ if $y_j(x) \geq y_k(x)\ \forall k \neq j$
- Total set of parameters

$$\tilde{W} = \begin{bmatrix} w_{1(D-1)} & w_{2(D-1)} & \cdots & w_{K(D-1)} \\ \vdots & \vdots & & \vdots \\ w_{10} & w_{20} & \cdots & w_{K0} \end{bmatrix}_{D\times K} \qquad \tilde{x} = \begin{bmatrix} x \\ 1 \end{bmatrix}$$

then we can define all y's as

$$y(x) = \tilde{W}^T\tilde{x} = \begin{bmatrix} y_1(x) \\ \vdots \\ y_K(x) \end{bmatrix}$$

- We use the least squares by transforming the target labels to their one hot encoded versions.
- We get the following results

$$E_D(\tilde{W}) = \frac{1}{2}Tr\{(\tilde{X}\tilde{W} - T)^T(\tilde{X}\tilde{W} - T)\}$$
$$\frac{\partial E}{\partial \tilde{W}} = 0 \implies \tilde{W} = (\tilde{X}^T\tilde{X})^{-1}\tilde{X}T = \tilde{X}^\dagger T$$

- **Logistic Regression** Reference: Chapter 4.2, PRML
- Consider the case of 2 classes

$$P(C_1|x) = \frac{P(x|C_1)PP(C_1)}{P(x|C_1)P(C_1) + P(x|C_2)P(C_2)} = \frac{1}{1 + \exp(-a)} = \sigma(a)$$

where $a = \ln\frac{P(x|C_1)P(C_1)}{P(x|C_2)P(C_2)}$

- σ is the logistic sigmoid function, whose range is $(0, 1)$

- Logistic regression tries to model $a = w^T x + b$ or $a = W^T \phi(x) + b$

$$p(C_1|\phi) = y(\phi) = \sigma(w^T \phi)$$

  Maximum likelihood solution, $\nabla E(w) = \sum_{n=1}^{N} (y_n - t_n)\phi_n$

- K-class logistic regression

$$p(C_k|\phi) = y_k(\phi) = \frac{\exp(a_k)}{\sum_j \exp(a_j)}, \ a_k = w_k^T \phi$$

  Maximum likelihood solution $\nabla_{w_j} E(w_1, ..., w_K) = \sum_{n=1}^{N} (y_{nj} - t_{nj})\phi_n$

- Since all probabilities are always in $[0, 1]$, the least squared distance is very small and is not robust.
- We need a better loss function.
- Consider the 2 class problem again, where $t_n = 1$ when $x_n \in C_1$ and 0 otherwise, we also have.

$$p(C_1|\phi) = y_n = \sigma(w^T x_n + b)$$

- When $t_n = 1$, we want $y_n^{t_n}$ to be higher and when $t_n = 0$, we want $(1 - y_n)^{1-t_n}$ to be higher. Jointly this can be written as

$$p(y_n) = y_n^{t_n}(1 - y_n)^{(1-t_n)}$$

- Each data sample is independent

$$p([y_1, y_2, ..., y_N]) = p(y_1)p(y_2)...p(y_N) = \prod_{n=1}^{N} y_n^{t_n}(1 - y_n)^{(1-t_n)}$$

- Log total probability

$$\log p([y_1, y_2, ..., y_N]) = \sum_{n=1}^{N} t_n \log y_n + (1 - t_n)\log(1 - y_n)$$

- We want to maximize the total probability or equivalently minimize the negative log probability(Binary Cross Entropy).
- The parameters of the model need to be solved based on the minimizing the error, however this error function is non-convex in the parameters.
- Need to perform iterative updates $\rightarrow$ Gradient descent.
- Consider a function $E[w]$, we assume that the derivative exists, then using the first order Taylor series approximation we can write

$$E[w + \epsilon] \approx E[w] + \epsilon^T \frac{\partial E}{\partial W}$$

  Let $\epsilon = -\eta \ \mathrm{sgn}(\frac{\partial E}{\partial w})$, where $\eta > 0$

$$E[w - \eta \ \mathrm{sgn}(\frac{\partial E}{\partial w})] \approx E[w] - \eta \ \mathrm{sgn}(\frac{\partial E}{\partial w})^T \frac{\partial E}{\partial W}$$

  Since, $\eta \ \mathrm{sgn}(\frac{\partial E}{\partial w})^T \frac{\partial E}{\partial W} \geq 0$

$$\implies E[w - \eta \ \mathrm{sgn}(\frac{\partial E}{\partial w})] \leq E[w]$$

- **Gradient Descent Algorithm**
- Reference: Deep Learning Book by Goodfellow. Chapter 4

- Consider we want to move from point $w \to w + u$, while minimizing some error function E, from taylor series expansion

$$E[w+u] \approx E[w] + u^T \frac{\partial E}{\partial w}$$
$$E[w] - E[w+u] = -u^T \frac{\partial E}{\partial w}$$

We want to maximize this value such that $u^T u = 1$

$$\implies \frac{\partial E}{\partial w} + 2\lambda u = 0$$
$$\implies u = -\frac{1}{2}\lambda \frac{\partial E}{\partial w} = -\eta \frac{\partial E}{\partial w}$$
$$w = w - \eta \frac{\partial E}{\partial w}$$

This is the method of steepest descent

This converges when $\frac{\partial E}{\partial w} \to 0$

- We can also use second order taylor series

$$E[w+u] \approx E[w] + u^T g + \frac{1}{2} u^T H u$$

Using the previous update rule, $u = -\eta g$

$$E[w - \eta g] = E[w] - \eta g^T g + \frac{1}{2}\eta^2 g^T H g$$

$\frac{1}{2}\eta^2 g^T H g$ is the curvature, correction due to hessian

Find u such that the above equation is maximized

$Hu = -g \implies u = -H^{-1} g$

This is Newton Method, much faster than gradient descent.

However, involves computation of inverse which is costly

- We can also make η adaptive, $\eta' = N\eta$

1. Start with random weights

$$iter\ t = 0 \to w^0, b^0$$

2. Compute gradients and update the model iteratively

$$w^t = w^{t-1} - \eta \frac{\partial E}{\partial w}\bigg|_{w,b=w^{t-1},b^{t-1}}$$
$$b^t = b^{t-1} - \eta \frac{\partial E}{\partial b}\bigg|_{w,b=w^{t-1},b^{t-1}}$$

3. Check for convergence
4. If not, $t = t + 1$ and go the step above

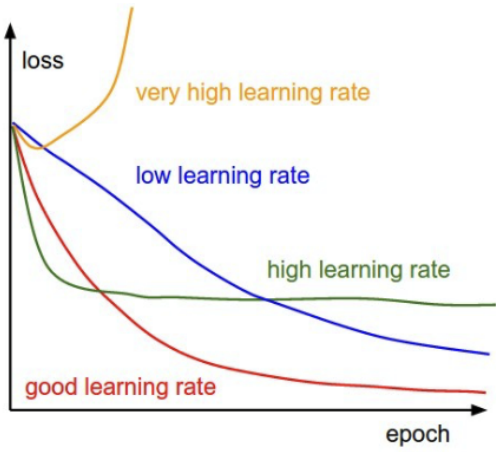


Figure 4: Choice of Learning Rate

- **Multi-Class Logistic Regression**
$$y = softmax(W^T x + b)$$
Targets are one hot encoded vectors, Error function is
$$E(W, b) = -\sum_n \sum_k t_{nk} \log y_{nk}$$
As before, parameters learnt using gradient descent.

- Taking the entire dataset and then updating might be too slow, requiring a large number of updates before convergence.

- **Stochastic Gradient Descent**: Split the data into random chunks, perform gradient updates on each mini batch

- One assumption we make above is that the gradients computed on the mini-batch well approximate the full batch gradients.

- This also introduces another hyperparameter, the batch size.

- **Kernel Machines**

- Reference: Chapter 6, PRML

- Recall Linear Models for Regression

- Consider the loss function
$$E(w) = \frac{1}{2}\sum_{n=1}^{N}(t_n - w^T\phi(x_n))^2 + \frac{\lambda}{2}w^T w$$
$$\frac{\partial E}{\partial w} = 0 \implies w = -\frac{1}{\lambda}\sum_{n=1}^{N}(w^T\phi(x_n) - t_n)\phi(x_n)$$
$$= \sum_{n=1}^{N} a_n \phi(x_n) = \Phi^T a$$
$\{a_n\}$ are dual variables
Substitute $w = \Phi^T a$
$$E(a) = \frac{1}{2}\sum_{n=1}^{N}(a^T\Phi\phi(x_n) - t_n)^2 + \frac{\lambda}{2}a^T\Phi\Phi^T a$$
$$= \frac{1}{2}a^T\Phi(\sum_{n=1}^{N}\phi(x_n)\phi(x_n)^T)\Phi^T a - a^T\Phi\sum_{n=1}^{N}\phi(x_n)t_n + \frac{1}{2}\sum_{n=1}^{N}t_n^2 + \frac{\lambda}{2}a^T\Phi\Phi^T a$$
$$= \frac{1}{2}a^T\Phi\Phi^T\Phi\Phi^T a - a^T\Phi\Phi^T t + \frac{1}{2}t^T t + \frac{\lambda}{2}a^T\Phi\Phi^T a$$

- Define $\kappa = \Phi\Phi^T_{N\times N}$, Gram Matrix
$$\kappa = \begin{bmatrix} \phi^T(x_1) \\ \phi^T(x_2) \\ \vdots \\ \phi^T(x_N) \end{bmatrix}_{N\times M} \begin{bmatrix} \phi(x_1) & \phi(x_2) & \cdots & \phi(x_N) \end{bmatrix}$$

- We define the kernel function as
$$[\kappa]_{m,n} = \phi^T(x_m)\phi(x_n) = k(x_m, x_n)$$
$$E[a] = \frac{1}{2}a^T\kappa\kappa^T a - a^T\kappa t + \frac{1}{2}t^T t$$
$$\frac{\partial E}{\partial a} = 0 \implies a = (\kappa + \lambda I_N)^{-1}t$$
$$y(x) = w^T\phi(x) = a^T\Phi\phi(x) = a^T \begin{bmatrix} k(x, x_1) \\ \vdots \\ k(x, x_N) \end{bmatrix}$$

- Datasets that are linearly separable with some noise work out great.
- But what to do if the dataset is just too hard, map it to a higher dimensional space.
- **General Idea**: The original input space can always be mapped to some higher dimensional feature space where the training set is separable
- A **kernel function** is some function that corresponds to an inner product in some expanded feature space.
- Consider a 2-dimensional vector $x = [x_1 \ x_2]$, let $\kappa(x_i, x_j) = (1 + x_i^T x_j)^2$

$$\begin{aligned}
\kappa(x_i, x_j) &= (1 + x_i^T x_j)^2 \\
&= 1 + x_{i1}^2 x_{j1}^2 + 2x_{i1}x_{j1}x_{i2}x_{j2} + x_{i2}^2 x_{j2}^2 + 2x_{i1}x_{j1} + 2x_{i2}x_{j2} \\
&= \begin{bmatrix} 1 & x_{i1}^2 & \sqrt{2}x_{i1}x_{i2} & x_{i2}^2 & \sqrt{2}x_{i1} & \sqrt{2}x_{i2} \end{bmatrix}^T \begin{bmatrix} 1 & x_{j1}^2 & \sqrt{2}x_{j1}x_{j2} & x_{j2}^2 & \sqrt{2}x_{j1} & \sqrt{2}x_{j2} \end{bmatrix} \\
&= \phi(x_i)^T \phi(x_j)
\end{aligned}$$

- **Mercer's Theorem**: Every semi-positive definite symmetric function is a kernel.
- Semi-positive definite symmetric functions correspond to a semi-positive definite symmetric Gram matrix.
- **Linear**: $\kappa(x_i, x_j) = x_i^T x_j$
- **Polynomial of power** p: $\kappa(x_i, x_j) = (1 + x_i^T x_j)^p$
- **Gaussian (radial-basis function network)**

$$\kappa(x_i, x_j) = \exp \frac{-\|x_i - x_j\|^2}{\sigma^2}$$

- **Sigmoid**: $\kappa(x_i, x_j) = \tanh(\beta_0 x_i^T x_j + \beta_1)$
- Properties of Kernel function
  k_1, k_2 are valid kernels, $c > 0$, $A \geq 0$ symmetric, and $q(\cdot)$ is a polynomial with coefficient ≥ 0

1. $k(x, x') = xk_1(x, x')$
2. $k(x, x') = f(x)k_1(x, x')f(x')$
3. $k(x, x') = q(k_1(x, x'))$
4. $k(x, x') = \exp(k_1(x, x'))$
5. $k(x, x') = k_1(x, x') + k_2(x, x')$
6. $k(x, x') = k_1(x, x')k_2(x, x')$
7. $k(x, x') = k_3(\phi(x), \phi(x'))$
8. $k(x, x') = x^T A x'$
9. $k(x, x') = k_a(x_a, x_a') + k_b(x_b, x_b')$
10. $k(x, x') = k_a(x_a, x_a')k_b(x_b, x_b')$

- **Maximum Margin Classifier**
- Define the **margin** of a linear classifier as the width that the boundary could be increased by before hitting a data point.
- Consider the line $y(x) = w^T \phi(x) + b$
- Normal distance or signed distance from any point to this line is $\frac{y(x)}{\|w\|}$
- Unsigned distance is $\frac{t_n y(x_n)}{\|w\|}$
- Maximum margin classifier, $\{w^*, b^*\}$

$$\arg\max_{\{w,b\}} \min_{x_n} \left\{ \frac{t_n(w^T \phi(x_n) + b)}{\|w\|} \right\} = \arg\max_{\{w,b\}} \min_{x_n} \left\{ \frac{\alpha}{\|w\|} \right\}$$

  $w \to \alpha w$, $b \to \alpha b$

- Choose α such that solution is unchanged

$$\arg\max_{w,b} \frac{1}{\|w\|} \text{ such that } t_n(w^T \phi(x_n) + b) \geq 1 \forall x_n$$

$$\arg\min_{w,b} \frac{1}{2}\|w\|^2 \text{ such that } t_n(w^T \phi(x_n) + b) \geq 1 \forall x_n$$

- Optimization rule for max-margin classifier: Quadratic function with linear inequality constraints.

$$L(w,b,a) = \frac{1}{2}w^T w - \sum_{n=1}^{N} a_n(t_n(w^T\phi(x_n)+b)-1)$$

$$\frac{\partial L}{\partial w} = 0 \implies w = \sum_{n=1}^{N} a_n t_n \phi(x_n)$$

$$\frac{\partial L}{\partial b} = 0 \implies \sum_{n=1}^{N} a_n t_n = 0$$

$$\begin{aligned}\tilde{L}(a) &= \frac{1}{2}(\sum_{m=1}^{N} a_m t_m \phi(x_m))(\sum_{n=1}^{N} a_n t_n \phi(x_n)) - \sum_{m=1}^{N} a_m t_m (\sum_{n=1}^{N} a_n t_n \phi(x_n)) \\ &= \sum_{n=1}^{N} a_n - \frac{1}{2}\sum_{n=1}^{N}\sum_{m=1}^{N} a_n a_m t_n t_m k(x_n, x_m)\end{aligned}$$

- Find $\{a_1, \cdots, a_N\}$ such that $a^* = \arg\max_a \tilde{L}(a)$ and $\sum_{n=1}^{N} a_n t_n = 0$, $a_n \geq 0\ \forall n$
- KKT conditions, $a_n(t_n(w^T\phi(x_n)+b)-1) = 0$
- **SVM Formulation**
  1. Correctly classify all training data

$$\begin{aligned} w^T\phi(x_n)+b \geq 1 \text{ if } t_n = +1 \\ w^T\phi(x_n)+b \leq 1 \text{ if } t_n = -1 \\ \implies t_n(w^T\phi(x_n)+b) \geq 1 \end{aligned}$$

  2. Define the Margin

$$\frac{1}{\|w\|}\min_n[t_n(w^T\phi(x_n)+b)]$$

  3. Maximize the Margin

$$\arg\max_{w,b}\left\{\frac{1}{\|w\|}\min_n[t_n(w^T\phi(x_n)+b)]\right\}$$

- Equivalently written as

$$\arg\min_{w,b}\frac{1}{2}\|w\|^2 \text{ such that } t_n(w^T\phi(x_n)+b) \geq 1$$

- **Constrained Optimization Basics**: Chapter 5 of `https://web.stanford.edu/~boyd/cvxbook/bv_cvxbook.pdf`
- Each non-zero a_n indicates that corresponding x_n is a support vector. Let S denote the set of support vectors

$$b = y(x_n) - \sum_{m\in S} a_m k(x_m, x_n)$$

- And the classifying function will have the form

$$y(x) = \sum_{n\in S} a_n k(x_n, x) + b$$

- **Overlapping Class Boundaries**
- The classes are not linearly separable, introducing slack variables $\zeta_n \geq 0$

$$t_n y(x_n) \geq 1 - \zeta_n$$

- The upper bound on mis-classification $\sum_n \zeta_n$
- The cost function to be optimized in this case

$$C\sum_n \zeta_n + \frac{1}{2}w^T w$$

- SVM formulation is very similar to previous case except for additional constraints

$$0 \leq a_n \leq C$$

$$\frac{\partial L}{\partial \zeta_n} = 0 \implies C = a_n + \mu_n = \mu_n = C - a_n$$

  where μ_n are the dual variables for the constraint $\zeta_n \geq 0$

- Solved using the dual formulation, sequential minimal optimization algorithm.

$$a_n < C \implies \zeta_n = 0 \text{ on the margin}$$

$$a_n = C \implies \zeta > 0 \implies \begin{cases} \zeta_n < 1, & \text{wrong side of margin but right side of decision boundary} \\ \zeta_n > 1, & \text{wrong side of margin and decision boundary} \end{cases}$$

$$a_n\}_{n=1}^{N} \to \begin{cases} a_n = 0, & \text{not a support vector} \\ 0 < a_n < C \implies \zeta_n = 0 & \text{but support vector on margin} \\ a_n = C \implies \zeta_n > 0 & \text{but violates margin} \end{cases}$$

- Solve b, pick all n such that $0 < a_n < C$
- Final classifier is based on the sign of

$$\sum_{n \in S} t_n a_n k(x_n, x) + b$$

- Properties of SVM
  1. Flexibility in choosing a similarity function
  2. Sparseness of solution when dealing with large data sets, only support vectors are used to specify the separating hyperplane.
  3. Overfitting can be controlled by soft margin approach
  4. A simple convex optimization problem which is guaranteed to converge to a single global solution
  5. Feature selection
- SVM has been used successfully in many real-world problems, text (and hypertext) categorization, image classification, bioinformatics(Protein classification, cancer classification), handwritten character recognition.
- SVMs are sensitive to noise, relatively small number of mislabeled examples can dramatically decrease the performance.
- SVMs only consider two classes, in case of multiclass we would have to build m different SVMs
- Choice of Kernel: Gaussian or polynomial kernel is default. If ineffective, more elaborate kernels are needed. Domain experts can give assistance in formulating appropriate similarity measures.
- Choice of kernel parameters: σ in Gaussian kernel, is the distance between the closest points with different classifications. In the absence of reliable criteria, applications rely on the use of a validation set or cross-validation to set such parameters.
- Optimization criterion: Hard margin vs Soft margin

4.4 Discriminative Modeling

- Multi-class logistic regression can be thought of as a single neuron or perceptron.
- Can we transform the data to linearly separable space, then apply the logistic regression to find the classifier.
- Can we learn such a transform from the data itself, non-linear transformation of the data needed.
- Can this also be realized as neural layer.
- Reference: Chapter 4, 6, 9, NNML
- Types of non-linearities
  1. **tanh**: Symmetric, $[-1, 1]$
  2. **sigmoid**: Continuous, positive valued

3. **ReLU**: $\max[0, x]$

- Using a softmax non-linearity, error function is cross entropy

$$E_{CE} = -\sum_n \sum_k t_{nk} \log y_{nk}$$

- For regression style tasks, output is linear, error function is mean square

$$E_{MSE} = \frac{1}{2} \sum_n \sum_k (t_{nk} - y_{nk})^2$$

- Computations in the forward direction

$$\begin{aligned} a^1 &= W^1 x + b^1 \\ z^1 &= \sigma(a^1) \\ a^2 &= W^2 z^1 + b^2 \\ y &= softmax(a^2) \end{aligned}$$

- Loss function is cross entropy
- Parameters in the model, $\Theta = \{W^1, b^1, W^2, b^2\}$
- Need to be updated based on the gradients w.r.t error
- When computing the gradients, order of computations
  1. The derivative of the loss function w.r.t output layer
  2. The derivative of the loss function w.r.t output activation
  3. The derivative of the loss function w.r.t hidden layer outputs
  4. The derivative of the loss function w.r.t hidden layer activations
- **Weight Decay Regularization**: Add $\frac{\lambda}{2}\|w\|_2^2$
- **Universal Approximation Theorem**: Any continuous and bounded mapping function can be ϵ approximated by a one-hidden neural network with sufficient number of hidden dimension and a squashing non-linearity.

$$\|\psi(x) - f(x)\|^2 < \epsilon$$

  $\psi(x)$ is function approximation or neural network.
- **Kolmogorov's Theorem**: Using sigmoidal non-linearity, $x \in \mathbb{R}^D$ can be guaranteed to approximate with $D(2D+1)$ hidden units.
- **Early Stopping**: Stop training when validation loss starts to increase for some time steps
- **Training with Noise**: $x \to x + \epsilon$, ϵ is random noise with low variance and uncorrelated to input.

$$\text{MSE Error} \implies E = \sum_{k=1}^{K} \int_x \int_t (y_k(x) - t_k)^2 P(x, t_k) dx dt_k$$

$$y_k(x+\epsilon) \approx y_k(x) + \sum_i \epsilon_i \frac{\partial y_k}{\partial x_i}\bigg|_{\epsilon=0} + O(\epsilon^2)$$

$$\tilde{E} = \sum_{k=1}^{K} \int_\epsilon \int_x \int_t (y_k(x+\epsilon) - t_k)^2 P(x, t_k) P(\epsilon) dx dt_k d\epsilon = E + \lambda\Omega \text{ (function of noise) + weight}$$

- **Committee of models (Ensembling)**: Assume you can train $j = 1, \cdots, Q$ neural networks.

$$y_j(x) = h(x) + e_j(x), \;\; h(x) = E[t|x], \;\; \text{where } t \text{ is one-hot encoded vector}$$

  Assume that error is MSE

$$\begin{aligned} E_j &= E[(y_j(x) - h(x))^2] = E[\epsilon_j^2] \\ E_{AV} &= \frac{1}{Q} \sum_{j=1}^{Q} E_j \end{aligned}$$

Committee rule $\leftarrow y_{COM}(x) = \frac{1}{Q}\sum_{j=1}^{Q} y_j(x)$

$$\begin{aligned} E_{COM} &= E[(y_{COM}(x) - h(x))^2] \\ &= E\left[\left(\frac{1}{Q}\sum_{j=1}^{Q} y_j(x) - h(x)\right)^2\right] \\ &= \frac{1}{Q^2}E[(\sum_{j=1}^{Q}\epsilon_j)^2] \end{aligned}$$

Assume ϵ_i, ϵ_j are uncorrelated, then $E_{COM} = \frac{1}{Q}E_{AV}$
From Cauchy's Inequality

$$E[(\sum_{i=1}^{L}\epsilon_i)^2] \leq E[(L\sum_{i=1}^{L}\epsilon_i^2)] \implies E_{COM} \leq E_{AV}$$

- **Mixture of Experts**: Can we do better than average, $y_{GEN} = \sum_{i=1}^{Q}\alpha_i y_i(x)$, where $0 \leq \alpha_i \leq 1$, $\sum_i \alpha_i = 1$

$$y_{GEN} = h(x) + \sum_{i=1}^{Q}\alpha_i\epsilon_i(x)$$

$$E_{GEN} = E[(y_{GEN}(x) - h(x))^2] = \sum_i\sum_j \alpha_i\alpha_j E(\epsilon_i\epsilon_j) = \sum_i\sum_j \alpha_i\alpha_j C_{ij}$$

Using KKT condition

$$L(\alpha) = E_{GEN} + \lambda(\sum_{i=1}^{Q}\alpha_i - 1)$$

$$\frac{\partial L}{\partial \alpha_i} = 0 \implies 2\sum_{j=1}^{Q}\alpha_j C_{ij} + \lambda = 0 \implies \alpha_i = \frac{\sum_{j=1}^{L}[C^{-1}]_{ij}}{\sum_k\sum_j[C^{-1}]_{kj}}$$

$$E_{GEN} = \alpha^T C\alpha = \frac{1}{\sum_m\sum_n[C^{-1}]_{mn}}$$

If $\alpha_i = \frac{1}{Q} \implies y_{GEN} = y_{AV}$

$$E_{GEN} \leq E_{COM} \leq E_{AV}$$

- Multi-layer Perceptron: Stack Perceptrons
- For complex problems in audio/image/text single hidden layer may be too restrictive in learning the model parameters. May not scale up with availability of big data.
- Deep networks perform **hierarchical data abstractions** which enable the non-linear separation of complex data samples.
- DNNs are quite data hungry, and performance improves by increasing the data.
- Generalization problem is tackled by providing training data from all possible conditions.
- Many artificial data augmentation methods have been successfully deployed.
- Providing the state-of-art performance in several real world applications.
- Representation Learning in Deep Networks
  1. The input data is one of the most important components of any machine learning systems.
  2. Extract factors that enable classification, while suppressing factors which are susceptible to noise.
  3. Finding the right representation for real world applications is substantially challenging.
  4. Deep Learning solution to this is to build complex representations from simpler representations.
  5. The dependencies between these hierarchical representations are refined by the target.
- Generally, depth improved the performance
- However, there are saturating effects of increasing the depth.

- **Momentum Gradient Descent**
  Intuition: If I am repeatedly being asked to move in the same direction then I should probably gain some confidence and start taking bigger steps in that direction. Just as a ball gains momentum while rolling down a slope.
  Update rule for momentum based gradient descent is

$$u_t = \beta u_{t-1} + \nabla w_t, \qquad u_0 = \nabla w_0,\ u_{-1} = 0$$
$$w_{t+1} = w_t - \eta u_t$$

  We are not only considering the current gradient, but we are also giving some importance to past history. β is typically less than 1, so we give decreasing importance to previous histories. We have

$$u_t = \beta u_{t-1} + \nabla w_t = \beta^2 u_{t-2} + \beta \nabla w_{t-1} + \nabla w_t = ... = \sum_{\tau=0}^{t} \beta^{t-\tau} \nabla w_\tau$$

  In addition to current update, also look at the history of updates.
  Even in the regions having gentle slopes, momentum based gradient descent is able to take large steps because the momentum carries it along.
  However, there is a possibility of overshooting our goal. This overshoot could lead to oscillations as well. Despite these oscillations, it will converge faster than gradient descent.

- **Nesterov Accelrated Gradient Descent**
  Can we do something to reduce the oscillations observed in momentum based gradient descent?
  Intuition: Look before you leap, we ask the weights to move by two parts βu_{t-1} and ∇w_t. The idea is to first move by βu_{t-1} and then compute ∇w_t. So instead of relying only on current gradient we are essentially "looking ahead" and computing new gradient.
  Update rule for NAG is

$$u_t = \beta u_{t-1} + \nabla(w_t - \beta u_{t-1})$$
$$w_{t+1} = w_t - \eta u_t$$
$$\text{with } u_{-1} = 0, \text{ and } 0 \leq \beta \leq 1$$

- Update rule for **AdaGrad**

$$v_t = v_{t-1} + (\nabla w_t)^2$$
$$w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t + \epsilon}} \times \nabla w_t$$

  and similar set of equations for b_t.

- **Intuition**: AdaGrad decays the learning rate very aggressively(as the denominator grows). As a result, after a while, the frequent parameters will start receiving very small updates. To avoid this we can decay the denominator.

- Update rule for **RMSprop**

$$v_t = \beta v_{t-1} + (1-\beta)(\nabla w_t)^2$$
$$w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t + \epsilon}} \times \nabla w_t$$

  Give lower and lower weightage to previous gradients

- RMSprop converges more quickly than AdaGrad by being less aggressive on decay.

- In RMSprop, the deltas start oscillating after a while, learning rate can increase, decrease or remain constant. Whereas in AdaGrad learning rate can only decrease as the denominator constantly grows.

- In case of oscillations, setting η_0 properly could solve the problem.

- Both are **sensitive** to initial learning rate, initial conditions of parameters and corresponding gradients.

- **AdaDelta** avoids setting initial learning rate η_0

$$v_t = \beta v_{t-1} + (1-\beta)(\nabla w_t)^2$$
$$\Delta w_t = -\frac{\sqrt{u_{t-1} + \epsilon}}{\sqrt{v_t + \epsilon}} \nabla w_t$$
$$w_{t+1} = w_t + \Delta w_t$$
$$u_t = \beta u_{t-1} + (1-\beta)(\Delta w_t)^2$$

Now the numerator, in the effective learning rate, is a function of past gradients. Observe that the u_t that we compute at t will be used only in the next iteration. Essentially one history runs behind the other.

- After some i iterations, v_t will start decreasing and the ratio of the numerator to the denominator starts increasing. If the gradient remains low for a subsequent time steps, then the learning rate grows accordingly.
- Therefore, AdaDelta allows the numerator to increase or decrease based on the current and past gradients.
- **Intuition**: Do everything that RMSprop does to solve the decay problem of AdaGrad, plus used a cumulative history of the gradients.
- Update rule for **Adam(Adaptive Moments)**

$$
\begin{aligned}
&m_t = \beta_1 m_{t-1} + (1-\beta_1)\nabla w,\ \hat{m}_t = \frac{m_t}{1-\beta_1^t} \leftarrow \text{Incorporating classical momentum} \\
&v_t = \beta_2 v_{t-1} + (1-\beta_2)(\nabla w_t)^2,\ \hat{v}_t = \frac{v_t}{1-\beta_2^t} \\
&w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t}+\epsilon}\hat{m}_t
\end{aligned}
$$

Note that we are taking a running average of the gradients as m_t.

- One way of looking at this is that we are interested in the expected value of the gradient and not on a single point estimate computed at time t.
- $\hat{m}_t$ and $\hat{v}_t$ are bias correction terms, we do this to avoid very high learning rate update. It can also be derived by taking expectation of m_t.
- Bias correction is the problem of exponential averaging.
- General form of L^p norm is $(|x_1|^p + |x_2|^p + ... + |x_n|^p)^{\frac{1}{p}}$, as $p \to \infty$, L^p boils down to taking the max value.
- In Adam, we called $\sqrt{v_t}$ as L^2 norm, why not replace it with L^∞ norm. Therefore, we have

$$
\begin{aligned}
&v_t = \max(\beta_2^{t-1}|\nabla w_1|, \beta_2^{t-2}|\nabla w_2|, ..., |\nabla w_t|) \\
&v_t = \max(\beta_2 v_{t-1}, |\nabla w_t|) \\
&w_{t+1} = w_t - \frac{\eta_0}{v_t}\nabla w_t
\end{aligned}
$$

Observe that we didn't use bias corrected v_t as max norm is not susceptible to initial zero bias.

- Suppose that we initialize w_0 such that the gradient at the w_0 is high. Suppose further that the gradients for the next subsequent iterations are also zero because x is sparse. Ideally, we don't want the learning rate to change its value when $\nabla w_t = 0$.
- Update Rule for **MaxProp** is similar to RMSprop

$$
\begin{aligned}
&v_t = \max(\beta v_{t-1}, |\nabla w_t|) \\
&w_{t+1} = w_t - \frac{\eta}{v_t+\epsilon}\nabla w_t
\end{aligned}
$$

- We can extend the same idea to Adam and call it **AdaMax**

$$
\begin{aligned}
&m_t = \beta_1 m_{t-1} + (1-\beta_1)\nabla e_t,\ \hat{m}_6 = \frac{m_t}{1-\beta_1^t} \\
&v_t = \max(\beta_2 v_{t-1}, |\nabla w_t|) \\
&w_{t+1} = w_t - \frac{\eta}{v_t+\epsilon}\hat{m}_t
\end{aligned}
$$

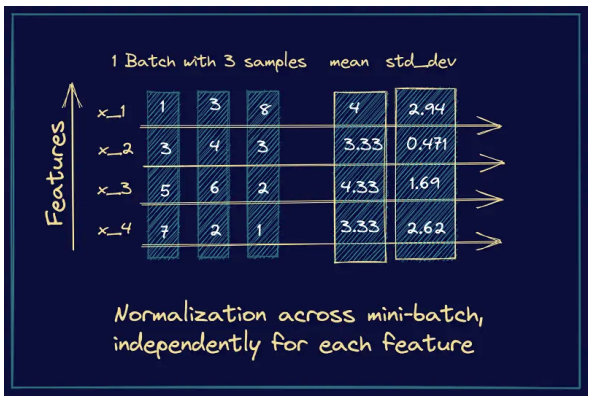
- **Intuition**: We know NAG is better than momentum based GD, why not just incorporate it with Adam.
- We can rewrite NAG such that there is no $t-1$ term, as

$$
\begin{aligned}
&g_{t+1} = \nabla w_t \\
&m_{t+1} = \beta m_t + \eta g_{t+1} \\
&w_{t+1} = w_t - \eta(\beta m_{t+1} + g_{t+1})
\end{aligned}
$$

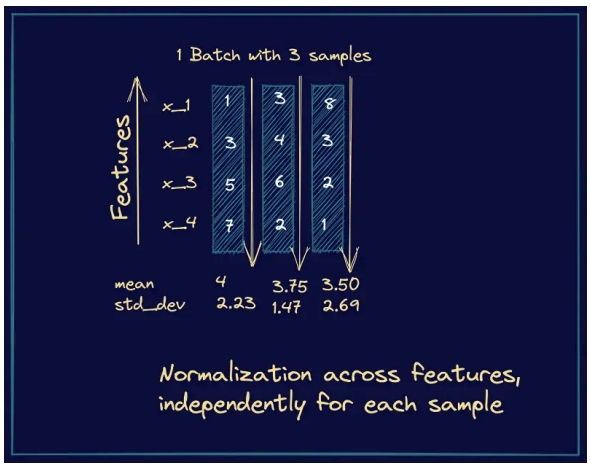
- Update rule for **NAdam(Nesterov Adam)**

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\nabla w,\ \hat{m}_t = \frac{m_t}{1-\beta_1^t}$$
$$v_t = \beta_2 v_{t-1} + (1-\beta_2)(\nabla w_t)^2,\ \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$
$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_{t+1}}+\epsilon}(\hat{m}_{t+1} + \frac{(1-\beta_1)\nabla w_t}{1-\beta_1^{t+1}})$$

- Reference: An overview of gradient descent optimization algorithms
- **Batch Gradient Descent**: Faster, Higher variance, convergence cannot be guaranteed
- **Learning Rate Schedulers**: As t increases, decrease η, convergence is improved.
- Learning rate choice is difficult.
- Read the paper and put stuff here
- **Dropout**: Random nodes are removed from the forward computation at dropout rate.
- Reference: Dropout: A Simple Way to Prevent Neural Networks from Overfitting
- Batch normalization normalizes each feature independently across the mini-batch.
- Layer normalization normalizes each of the inputs in the batch independently across all features
- As batch normalization is dependent on batch size, it's not effective for small batch sizes.
- Layer normalization is independent of the batch size, so it can be applied to batches with smaller sizes as well.
- Batch normalization requires different processing at training and inference times.
- As layer normalization is done along the length of input to a specific layer, the same set of operations can be used at both training and inference times.
- Reference: Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift



(a) Batch Normalization



(b) Layer Normalization

- **Loss function**: Goal is for NN output to behave like a class posterior, $y_k = P(C_k|x)$
  Let us call a general continuous and differentiable function f, $f(y_k, t_k) = f(|y_k - t_k|)$

$$\text{Expected Error } \langle E \rangle = \sum_{k=1}^{C} \int_{t_k} \int_x f(|y_k - t_k|) P(t_k|x) P(x) dx dt_k$$
$$= \sum_{k=1}^{C} \left\{ \int_x f(1-y_k) P(C_k|x) P(x) dx + \int_c f(y_k)(1 - P(C_k|x) P(x) \right\}$$
$$\frac{\partial \langle E \rangle}{\partial y_k(x)} = 0 \implies -f'(1-y_k)P(C_k|x) + f'(y_k)(1 - P(C_k|x)) = 0$$
$$\implies \frac{f'(1-y_k)}{f'(y_k)} = \frac{1-P(C_k|x)}{P(C_k|x)} = \frac{1-y_k}{y_k}$$

The class of functions that satisfy this, $f(y) = \int y^\tau (1-y)^{\tau-1} dy$, this satisfies $\frac{f'(1-y)}{f'(y)} = \frac{1-y}{y}$

$$\tau = 1 \to f(y) = \frac{y^2}{2}$$
$$f(|t_k - y_k|) = \frac{|t_k - y_k|^2}{2} = \text{MSE}$$

$$\tau = 0 \to f(y) = -\ln(1-y)$$
$$f(|y-t|) = -\ln(1-|y-t|) = -t\ln y + (1-t)\ln(1-y) = \text{BCE}$$

4.5 Neural Network Architectures

- DNNs are suboptimal for images because they ignore the local correlations in the pixels, and the geometric structure is not exploited in images.
- Convolution operation
$$A(i,j) = \sum_{m=0}^{M}\sum_{n=0}^{N} W(m,n)X(i+m, j+n)$$
- Derivative of loss wrt weights in a convolutional layer
$$\begin{aligned}
&A(0,0) = X(0,0)W(0,0) + X(0,1)W(0,1) + \ldots + X(M-1,N-1)W(M-1,N-1)\\
&A(0,1) = X(0,1)W(0,0) + X(0,2)W(0,1) + \ldots\\
&\frac{\partial L}{\partial W}(0,0) = \frac{\partial L}{\partial A}(0,0)X(0,0) + \frac{\partial L}{\partial A}(0,1)X(0,1) + \ldots\\
&\frac{\partial L}{\partial W}_{M\times N} = X_{H\times W} \circledast \frac{\partial L}{\partial A}_{(H-M+1)\times(W-N+1)}
\end{aligned}$$
- Reduce the size of images after convolution using pooling.
- Keep local maximum, minimum, or average.
- Multiple levels of filtering and subsampling operations.
- Feature maps are generated at every layer.
- **Backforward for Pooling**: Consider a Max pooling layer for a 2×2 section, whatever max value we got in forward propagation, we just make a 2×2 matrix of the same values for backpropoation.
- Reduce number of parameters due to weight sharing.
- Depth does not necessarily increase the parameter size.
- CNN filters operate on local weights.
- Deeper layers capture wider input context.
- Training is more memory intensive, accumulate gradients.
- Reference: Visualizing and Understanding Convolutional Networks
- Analyze a trained model
  1. Take a model which is trained to perform object classification
  2. Map the layer outputs back to the original space of images
  3. Analyze the reconstruction outputs on a held out data
- Model learns a hierarchy
- Earlier layer perform simpler tasks like edge detection.
- Final layers perform complex tasks like object parts.
- Deep networks perform hierarchical data abstraction.
- Now we move to sequence data

- $(x(t), y(t))$ where this could be a sequence to sequence mapping task
  $(x(t), y)$ where this could be a sequence to vector mapping task
  Input features are correated in time.

- We need to model dependenices in the sequence over time.

- **First Order Recurrence**: Make the hidden layer a function of the previous outputs from the hidden layer along with the input
$$h(t) = f(h(t-1), x(t))$$
  This makes the hidden layer dependent of previous layer outputs in a recurring fashion.

- Model Forward Pass for 1 hidden layer
$$\begin{aligned}
a^1(t) &= W^1 x(t) + U^1 h^1(t-1) \\
h^1(t) &= tanh(a^1(t)) \\
a^2(t) &= W^2 h^1(t) \\
\hat{y}(t) &= Softmax(a^2(t))
\end{aligned}$$
  Ignore bias for simplicity

- Error functions are computed at every time-instant, total error $E = \sum_{t=1}^{T} E(y(t), \hat{y}(t))$

- To backpropaget, we define **Back Propagation through Time (BPTT)**
$$\begin{aligned}
\frac{\partial E}{\partial W^2}(t) &= h^1(t)^T \frac{\partial E}{\partial a^2}(t) \quad \forall t \\
\frac{\partial E}{\partial W^1}(t) &= \left[(1 - h^1(t)^2) \circ W^{2T} \frac{\partial E}{\partial a^2(t)}\right] \left[x(t)^T + U^{1T}\left(\frac{\partial h^1(t-1)}{\partial W^1}\right)\right] \quad \forall t \geq 0 \\
\frac{\partial h^1(t)}{\partial W^1} &= (1 - h^1(t)^2) \circ \left[x(t)^T + U^{1T}\left(\frac{\partial h^1(t-1)}{\partial W^1}\right)\right] \quad \forall t \geq 0 \\
\frac{\partial E}{\partial U^1}(t) &= \left[(1 - h^1(t)^2) \circ W^{2T} \frac{\partial E}{\partial a^2(t)}\right] \left[h^1(t-1)^T + U^{1T} \frac{\partial h^1(t-1)}{\partial U^1}\right] \quad t \geq 0 \\
\frac{\partial h^1(t)}{\partial U^1} &= (1 - h^1(t)^2) \circ \left[h^1(t-1) + U^{1T} \frac{\partial h^1(t-1)}{\partial U^1}\right] \quad t \geq 0
\end{aligned}$$

- This, however has long term dependency issues, i.e., gradients tend to vanish or explode

- **:** Define three gates, input, forget and output.
$$\begin{aligned}
f(t) &= \sigma(W^f x(t) + U^f h(t-1)) \\
i(t) &= \sigma(W^i x(t) + U^i h(t-1)) \\
o(t) &= \sigma(W^o x(t) + U^o h(t-1)) \\
a^1(t) &= W^1 x(t) + U^1 h^1(t-1) \\
\tilde{c}^1(t) &= tanh(a^1(t)) \\
c^1(t) &= f_t \circ c(t-1) + i_t \circ \tilde{c}^1(t) \\
h^1(t) &= o(t) \circ tanh(c^1(t)) \\
a^2(t) &= W^2 h^1(t) \\
\hat{y}(t) &= Softmax(a^2(t))
\end{aligned}$$

- **Attention**: Suppose we pass the input to the network and get hidden states, $h(1), h(2), \ldots h(T)$. Attention is simply a weighted average of these states which would be learned from data
$$\begin{aligned}
u_t &= tanh(W_l h_t + b_l) \\
a_t &= \frac{\exp(u_t^T u_l)}{\sum_t \exp(u_t^T u_l)} \\
l &= \sum_t a_t h_t \\
\hat{y} &= Softmax(W_q l)
\end{aligned}$$

- Certain regions of the input have more importance than the rest of the task at hand. Attention allows a mechanism to add relevance.
- **Natural Language Processing**: To use these models on language, we are required to convert text into fixed representations in a vector space.
- Using one-hot encoded vectors is really high dimensional, need a concise word representations that embed semantics.
- **Word2Vec Models**: Models to learn text representation, two simple models are continuous bag of words and skip-gram
- **CBOW**: Given input of n words behind t and n words ahead of t predict the word at t.
  We begin by one-hot encoding all words.
  Suppose mth position has 1, so when we multiply the word vector with the weight vector, we are essentially taking the mth row.
  Idea is that after training, each row of the weight matrix becomes the vector representation of the corresponding word.
- **Skip-Gram**: Given a word at t, predict n words ahead of it and behind it.

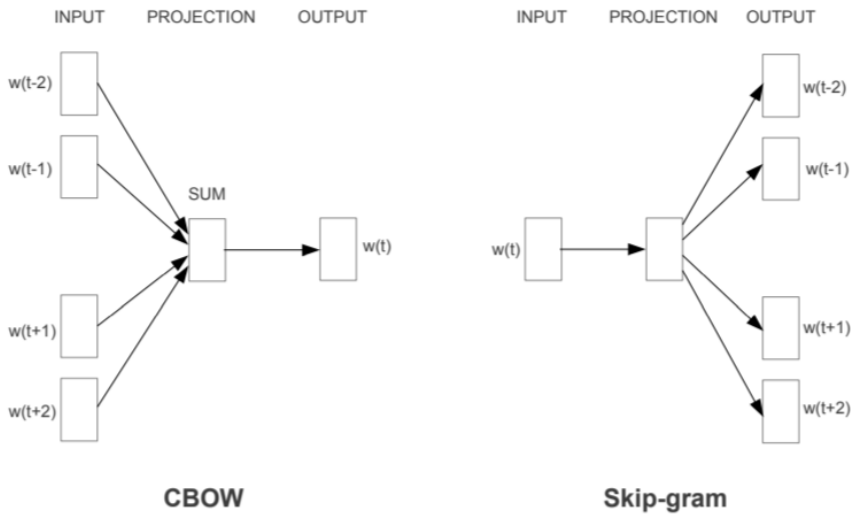


Figure 6: Word2Vec comparison

- **Self-Attention**: Suppose we have T inputs of D dimension, self-attention involves three learnable matrices via which we define key, value, and query.

$$
\begin{aligned}
X &= \{x_1, x_2, \ldots, x_T\}; x_t \in \mathbb{R}^D \\
K &= XW_k = \{k_1, k_2, \ldots, k_T\} \in \mathbb{R}^D \\
Q &= XW_q = \{q_1, q_2, \ldots, q_T\} \in \mathbb{R}^D \\
V &= XW_v = \{v_1, v_2, \ldots, v_T\} \in \mathbb{R}^D
\end{aligned}
$$

  Using key and query, we calculate attention values

$$a = Softmax\left(\frac{QK^T}{\sqrt{D}}\right)$$

  and using the attention values, we take the weighted average of value, which forms our output for self attention

$$= aV$$

- Normally, we will have multiple self attentions in a layer, which will be called multi headed self attention

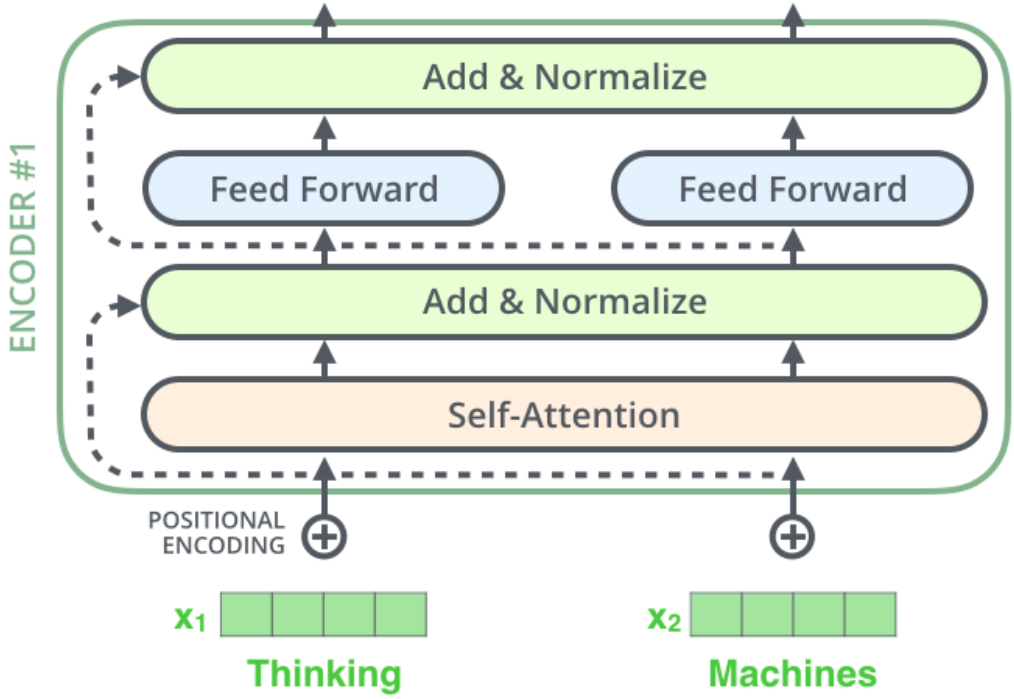


Figure 7: Transformer Encoder

- **Encoder-Decoder Attention**: It is basically the same as self attention, except the key and value matrices are formed using encoder output and query is formed using decoder input.

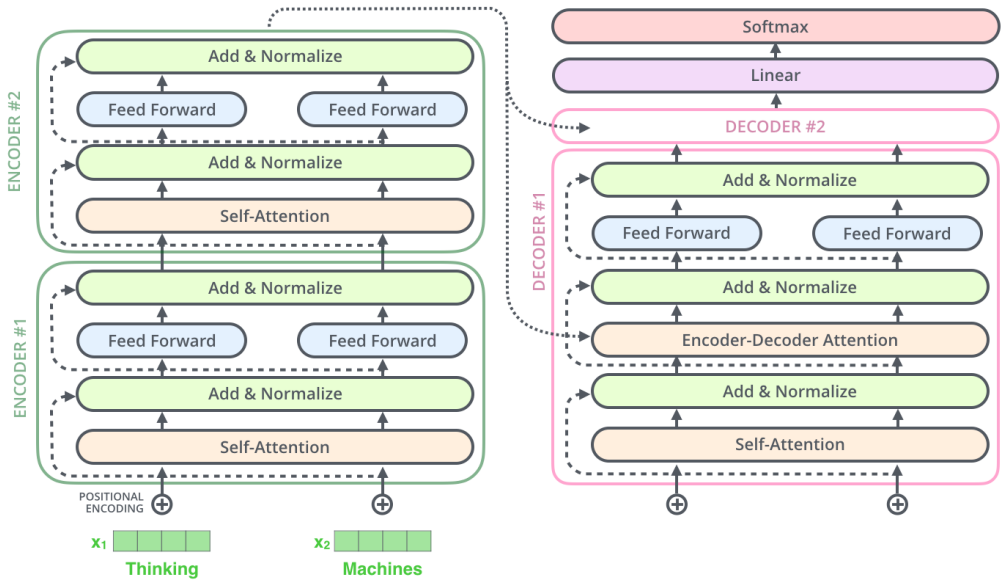


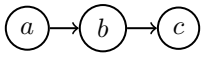
Figure 8: Transformer Decoder

- Reference: `https://jalammar.github.io/illustrated-transformer/`
- **Unsupervised Learning**: Developing models that do not need labels. May model the generation of data or may allow generation of new data samples.
- **Self-supervision**: Different from supervised and unsupervised learning.
  Does not perform distribution learning or reconstruction
  se a pretext task. Performing contrastive or predictive learning.
  Using large volumes of unsupervised data
- Two levels of modeling with unsupervised data: Firs generate a psuedo label and then learn the upstream model

- Start with unlabeled data, generate psudeo labels and use the psudeo labels to pre-train the model.
- Then the learned weights of feature etractor are used to solve some downstream task.
- Reference: Self-supervised representation learning: Introduction, advances, and challenges
- For pre-training, one way to do it is to mask portions of the input and pass the rest of the embeddings with zero or random entries at the masked locations to the transformer encoder.
- **Masked Language Modelling(MLM)**: Have the model predict the word tokens in the masked portions.
- BERT models use MLM pre-training
- **Large Language Models (LLMs)**: Extends the task of self-supervision.
- First, we mine lots of text data and design the model with large capacity, i.e., billions of parameters.
- Then we pre-train the model with MLM and similar style of losses, this requires high resource of computations
- Pre-training can be thought of as making the model understand language and grammer.
- Final trained model can be fine-tuned for supervised tasks.
- Self-supervised learning has shown emergent abilities to generalize to wide variety of downstream tasks.
- It works well on tasks that the model was not trained on, which is not seen in smaller models. This enables to build reasoning capabilities in the model.
- Some Future works could involve the following
- **Multi-modal**: Incorporate learning across modalities, create a domain specific encoder/decoder and learning the joint language model
- Combining some labeled data with the self-supervised data to further improve the models, current models like GPT use human feedback aka RLHF.

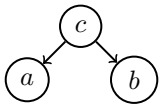
5 Graphical Models

- Reference: Chapter 8 of PRML
- Markov random fields can be represented as undirected graphs
- Bayesian Networks can be represented as directed graphs, or more specifically directed acyclis graphs.



$$P(a,b,c) = P(a)P(b|a)P(c|b)$$

- All nodes or vertices are random variables
- All edges represent conditional dependence
- We can define three types of conditional independence
- **Tail-to-Tail Connection**



$$\begin{aligned}
&P(a,b,c) = P(c)P(b|c)P(a|c) \\
&P(a,b|c) = \frac{P(a,b,c)}{P(c)} = P(b|c)P(a|c) \implies a \amalg b|c \\
&P(a,b) = \sum_c P(a|c)P(b|c)P(c) \implies a \not\amalg b|\phi
\end{aligned}$$

- **Head-to-Tail Connection**

$$a \to c \to b$$

$$P(a,b,c) = P(a)P(c|a)P(b|c)$$

$$P(a,b|c) = \frac{P(a,b,c)}{P(c)} = P(a|c)P(b|c) \implies a \perp\!\!\!\perp b|c$$

$$P(a,b) = \sum_c P(a)P(c|a)P(b|c) \implies a \not\perp\!\!\!\perp b|\phi$$

- **Head-to-Head Connection**

$$\begin{tikzcd} a \arrow[dr] & & b \arrow[dl] \\ & c & \end{tikzcd}$$

$$P(a,b,c) = P(a)P(b)P(c|a,b)$$

$$P(a,b|c) = \frac{P(a,b,c)}{P(c)} \implies a \not\perp\!\!\!\perp b|c$$

$$P(a,b) = \sum_c P(a)P(b)P(c|a,b) = P(a)P(b) \implies a \perp\!\!\!\perp b|\phi$$